
Rapport de Mini-Projet

Gestion Bibliothèque

Réalisé par : Israe Tchich

Imane Mesrar

Encadrante : Mme Douae EL HILA

Module : Développement Java IHM

Année : 2025/2026

Remerciements

Nous tenons à remercier notre encadrante Mme Douae EL HILA pour nous avoir offert l'opportunité de réaliser ce mini-projet dans le cadre du module Développement Java IHM.

Nous la remercions également pour l'encadrement assuré durant cette année universitaire et pour les connaissances transmises à travers ce module.

Nous remercions aussi l'École Nationale des Sciences Appliquées d'Oujda ainsi que l'ensemble des enseignants qui ont contribué à notre formation.

Enfin, nous remercions toutes les personnes ayant participé, de près ou de loin, à la réalisation de ce travail.

Table des matières

Remerciements.....	2
Introduction.....	6
Présentation du projet.....	7
1.1 Description de l'application.....	7
1.2 Besoins fonctionnels.....	7
1.3 Besoins non fonctionnels.....	8
Conception.....	9
2.1 Architecture MVC.....	9
2.2 Diagramme de classes UML.....	10
2.3 Diagramme de cas d'utilisation.....	11
Environnement de travail.....	12
3.1 Outils et technologies utilisés.....	12
3.2 Structure du projet.....	13
Répartition des tâches.....	14
4.1 Tâches de l'étudiant 1 – Tchich Israe.....	14
4.2 Tâches de l'étudiant 2 – Mesrar Imane.....	14
Explication du code.....	15
5.1 Connexion à la base de données – Database.java.....	15
5.2 Les classes modèles.....	15
5.3 Les classes DAO.....	15
5.4 Les contrôleurs.....	16
5.5 Les fichiers FXM.....	16
5.6 Fonctionnalité spécifique au projet.....	16
Interfaces de l'application.....	18
6.1 Fenêtre principale.....	18
6.2 Interface de gestion des livres.....	18
6.3 Interface d'ajout et de modification.....	19
6.4 Affichage des donnée.....	19
6.5 Menus et dialogues.....	20
Conclusion.....	21
Références.....	22

Table des figures

Figure 1 : Architecture MVC de l'application	9
Figure 2 : Diagramme de classes UML	10
Figure 3 : Diagramme de cas d'utilisation	11
Figure 4 : Structure du projet dans IntelliJ IDEA	13
Figure 5 : Fenêtre principale de l'application	18
Figure 6 :Interface de gestion des livres	19
Figure 7 : Formulaire d'ajout et de modification	19
Figure 8 : Tableau d'affichage des données	20
Figure 9 : Fenêtre de dialogue de l'application	20

Liste des tableaux

4.1 Tâches réalisées par l'étudiant 1	14
4.2 Tâches réalisées par l'étudiant 2	14

Introduction

Dans le cadre du module Développement Java IHM, ce mini-projet a pour objectif de mettre en pratique les connaissances acquises en programmation orientée objet, en développement d'interfaces graphiques avec JavaFX ainsi qu'en gestion des bases de données avec JDBC et MySQL.

L'objectif principal de ce projet est de concevoir et développer une application permettant de gérer des informations de manière simple, organisée et efficace.

Pour assurer une bonne organisation du code, nous avons adopté une architecture MVC (Model View Controller) qui permet de séparer la partie interface utilisateur, la logique de traitement et la gestion des données.

Ce rapport présente les différentes étapes de réalisation de notre application, en commençant par la présentation du projet et l'analyse des besoins, puis la conception avec les diagrammes UML, l'environnement de travail utilisé, l'explication du code et enfin les interfaces réalisées.

Chapitre 1

Présentation du projet

1.1 Description de l'application

L'application développée dans le cadre de ce mini-projet est une application de gestion réalisée avec JavaFX pour la conception de l'interface graphique et JDBC pour assurer la communication avec une base de données MySQL.

Cette application permet à l'utilisateur de gérer les différentes informations liées au domaine du projet à travers une interface simple et intuitive.

L'objectif principal est de proposer une solution informatisée permettant de faciliter la gestion des données, d'effectuer les opérations nécessaires et d'assurer une meilleure organisation des informations.

La conception de l'application repose sur une architecture MVC (Model View Controller) afin de séparer les différentes parties du programme et faciliter la maintenance du code.

1.2 Besoins fonctionnels

Les besoins fonctionnels représentent les différentes fonctionnalités que l'application doit fournir à l'utilisateur.

L'application doit permettre :

- L'ajout de nouvelles informations.
- La modification des données existantes.
- La suppression des données.
- L'affichage des informations enregistrées.
- La recherche et la consultation des éléments selon les besoins du projet.
- L'interaction avec l'application à travers une interface graphique.

1.3 Besoins non fonctionnels

Les besoins non fonctionnels représentent les contraintes liées à la qualité et au fonctionnement de l'application.

L'application doit respecter les critères suivants :

- Une interface facile à utiliser.
- Une bonne organisation du code grâce à l'architecture MVC.
- Une connexion fiable avec la base de données.
- Une bonne performance lors de l'exécution des opérations.
- Une facilité de maintenance et d'évolution du projet.

Chapitre 2

Conception

2.1 Architecture MVC

L'architecture MVC (Model View Controller) est un modèle de conception permettant de séparer l'application en trois parties principales : le modèle, la vue et le contrôleur.

Cette organisation permet de rendre le code plus clair, plus facile à maintenir et à faire évoluer.

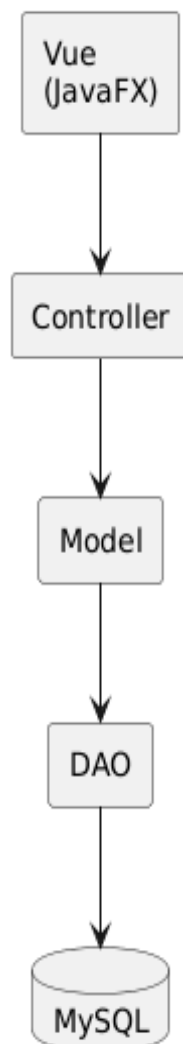


Figure 1 : Architecture MVC de l'application

2.2 Diagramme de classes UML

Le diagramme de classes représente la structure de l'application. Il présente les classes utilisées, leurs attributs, leurs méthodes ainsi que les relations entre elles.

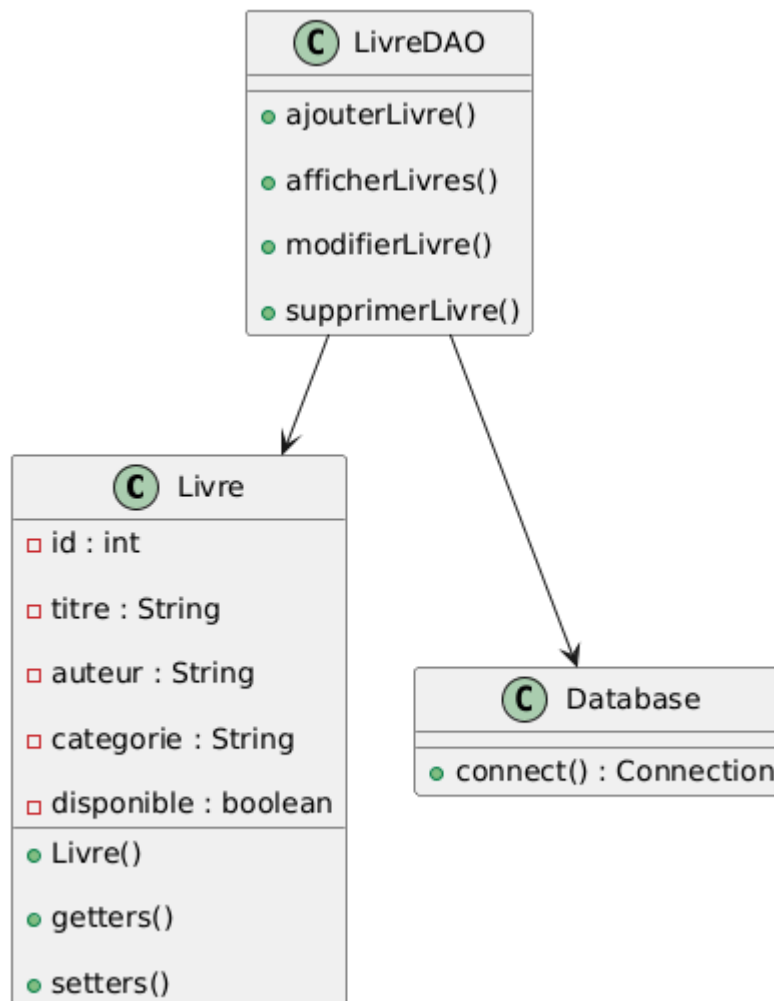


Figure 2 : Diagramme de classes UML

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les interactions entre l'utilisateur et les fonctionnalités principales offertes par l'application

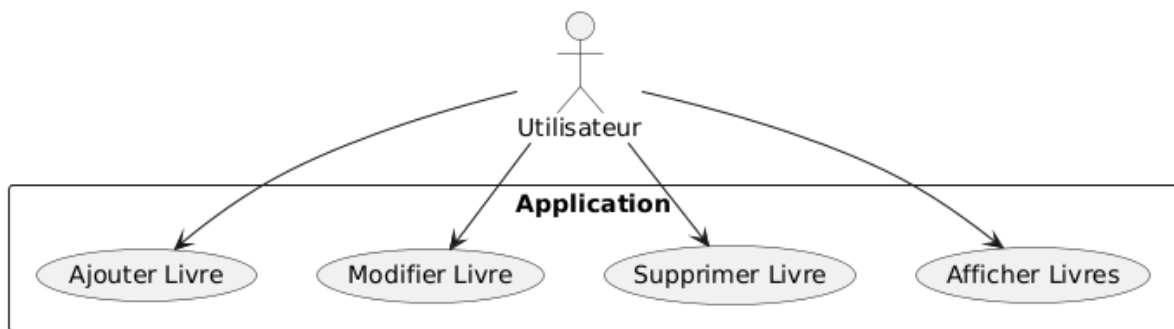


Figure 3 : Diagramme de cas d'utilisation UML

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

Pour la réalisation de cette application, plusieurs outils et technologies ont été utilisés afin d'assurer le développement de l'interface graphique, la gestion des données et l'organisation du projet.

- Java :

Java est le langage de programmation utilisé pour développer l'application. Il permet de mettre en œuvre la programmation orientée objet.

- JavaFX :

JavaFX est une bibliothèque permettant de créer des interfaces graphiques modernes et interactives. Elle est utilisée pour réaliser les différentes fenêtres et composants de l'application.

- JDBC :

JDBC (Java Database Connectivity) est une API Java permettant d'établir la communication entre l'application et la base de données.

- MySQL :

MySQL est le système de gestion de base de données utilisé pour stocker et gérer les informations de l'application.

- IntelliJ IDEA :

IntelliJ IDEA est l'environnement de développement utilisé pour écrire, exécuter et tester le code Java.

- GitHub :

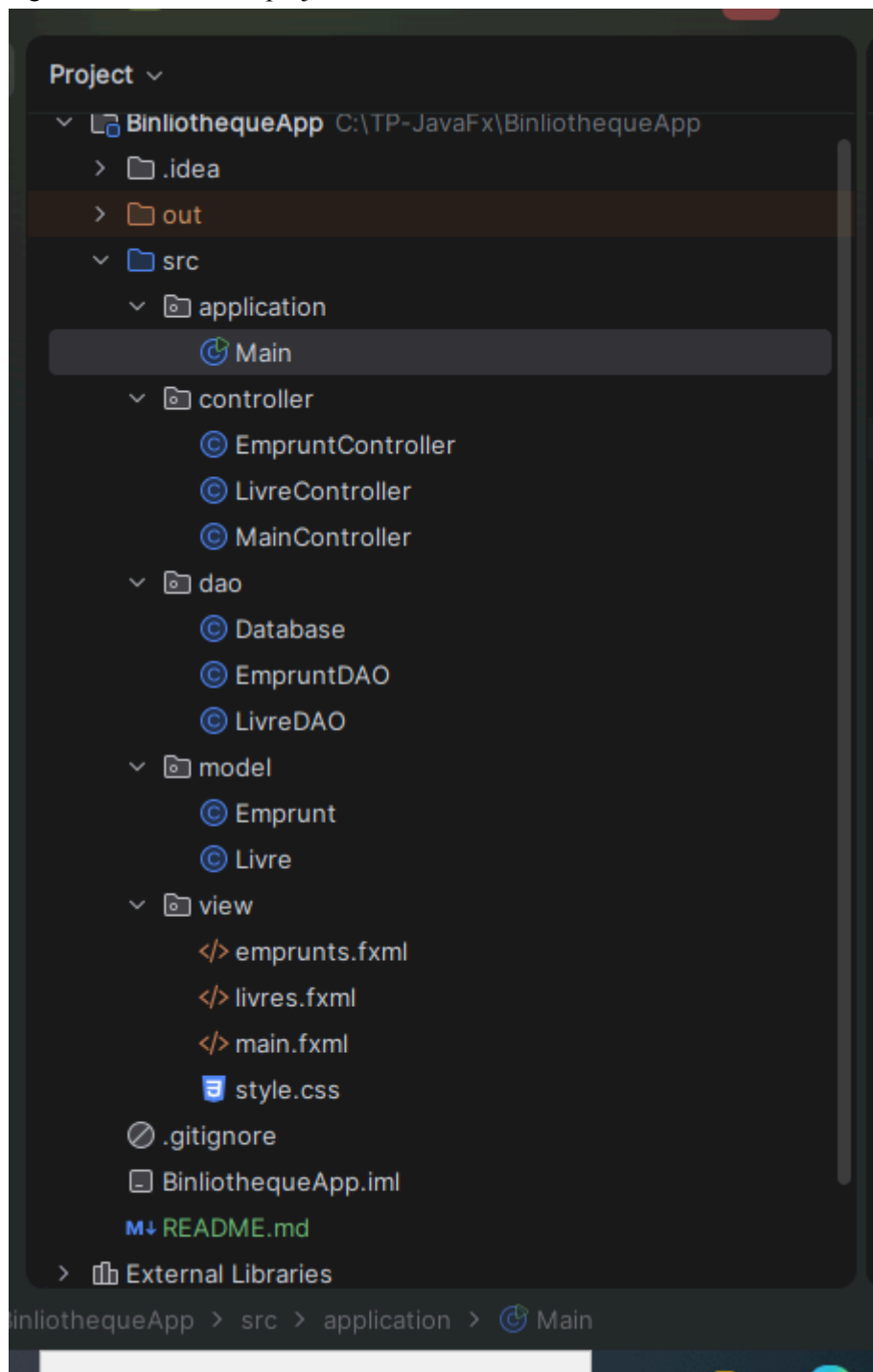
GitHub est utilisé pour faciliter la gestion du projet et le partage du code entre les membres de l'équipe.

3.2 Structure du projet

L'application est organisée selon une structure permettant de séparer les différentes parties du programme. Cette organisation facilite la compréhension, la maintenance et l'évolution du code.

La structure principale du projet est la suivante :

Figure 4 : Structure du projet dans IntelliJ IDEA



Chapitre 4

Répartition des tâches

4.1 Tâches de l'étudiant 1 – Tchich Israe

Tâche réalisée	Fichier(s) concerné(s)	Statut
Gestion de la base de données	MySQL / Database.java	Terminé
Création des classes DAO	Classes DAO	Terminé
Mise en place de la connexion JDBC	Database.java	Terminé
Gestion des opérations CRUD	DAO / Requêtes SQL	Terminé

Table 4.1 – Tâches réalisées par Tchich Israe

4.2 Tâches de l'étudiant 2 – Mesrar Imane

Tâche réalisée	Fichier(s) concerné(s)	Statut
Création des interfaces graphiques JavaFX	Fichiers FXML / View	Terminé
Développement des contrôleurs	Classes Controller	Terminé
Gestion des interactions utilisateur	Controller.java	Terminé
Tests et vérification des fonctionnalités	Projet JavaFX	Terminé

Table 4.2 – Tâches réalisées par Mesrar Imane

Chapitre 5

Explication du code

5.1 Connexion à la base de données – Database.java

La classe Database permet d'établir la connexion entre l'application Java et la base de données MySQL.

Elle utilise JDBC (Java Database Connectivity) afin de communiquer avec la base de données.

Cette classe contient une méthode connect() qui permet de créer une connexion grâce aux informations de connexion comme l'adresse de la base de données, le nom d'utilisateur et le mot de passe.

Cette organisation permet de centraliser la gestion de la connexion et de la réutiliser dans les autres parties de l'application.

5.2 Les classes modèles

Les classes modèles représentent les différentes entités manipulées par l'application.

Elles contiennent les attributs correspondant aux données enregistrées dans la base de données ainsi que les constructeurs, getters et setters nécessaires pour accéder et modifier ces informations.

Ces classes permettent de représenter les objets de l'application et facilitent la communication entre les différentes couches du programme.

5.3 Les classes DAO

Les classes DAO (Data Access Object) sont responsables de la gestion des opérations avec la base de données.

Elles contiennent les méthodes permettant d'effectuer les différentes opérations CRUD :

- Ajouter des données (INSERT).
- Modifier des données (UPDATE).
- Supprimer des données (DELETE).
- Récupérer les données (SELECT).

L'utilisation des DAO permet de séparer la gestion des données de la logique de l'application.

5.4 Les contrôleurs

Les contrôleurs assurent la liaison entre l'interface graphique et le reste de l'application.

Ils récupèrent les actions effectuées par l'utilisateur, comme les clics sur les boutons, puis exécutent les traitements nécessaires.

Dans l'architecture MVC, les contrôleurs jouent le rôle d'intermédiaire entre la Vue et le Modèle.

5.5 Les fichiers FXM

Les fichiers FXML sont utilisés pour décrire la structure des interfaces graphiques JavaFX.

Ils permettent de définir les différents composants de l'interface comme les boutons, champs de saisie, tableaux et fenêtres.

Cette séparation entre l'interface graphique et le code Java améliore l'organisation et facilite la modification de l'apparence de l'application.

5.6 Fonctionnalité spécifique au projet

La fonctionnalité principale de notre application est la gestion des livres.

Elle permet à l'utilisateur d'ajouter, modifier, supprimer et consulter les informations des livres enregistrés.

Cette fonctionnalité est réalisée grâce à l'interaction entre l'interface JavaFX, les contrôleurs, les classes modèles et les classes DAO qui assurent la communication avec la base de données.

Chapitre 6

Interfaces de l'application

6.1 Fenêtre principale

La fenêtre principale représente l'interface d'accueil de l'application. Elle permet à l'utilisateur d'accéder aux différentes fonctionnalités proposées.

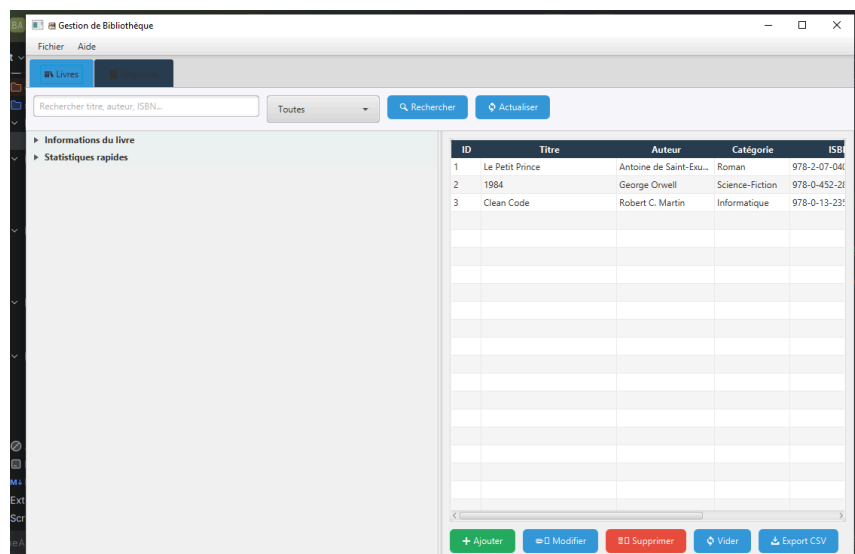


Figure 5 : Fenêtre principale de l'application

6.2 Interface de gestion des livres

Cette interface permet à l'utilisateur de gérer les informations liées aux livres. Elle permet d'effectuer les opérations d'ajout, modification, suppression et consultation.

ID	Livre	Emprunteur	Date emprunt	Retour prévu	Statut	
1	Le Petit Prince	Antoine	2026-06-19	2026-07-03	En cours	

Figure 8 : Tableau d’affichage des données

6.5 Menus et dialogues

L’application contient des messages et des fenêtres de dialogue permettant d’informer l’utilisateur lors des différentes opérations réalisées.

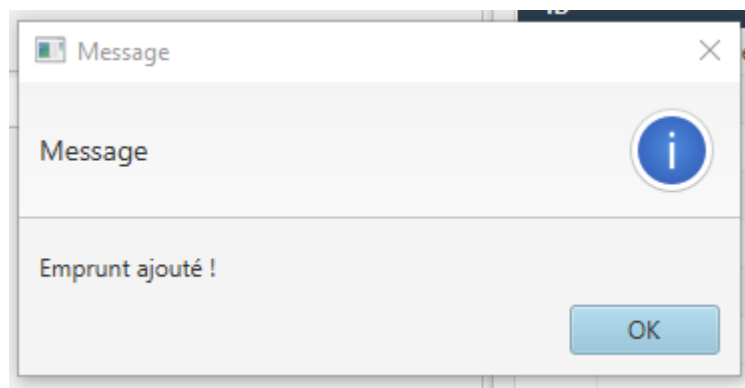


Figure 9 : Fenêtre de dialogue de l’application

Conclusion

Ce mini-projet nous a permis de mettre en application les connaissances acquises dans le domaine du développement Java IHM, notamment la programmation orientée objet, la création d'interfaces graphiques avec JavaFX et la gestion des données avec JDBC et MySQL.

La réalisation de cette application nous a permis de comprendre les différentes étapes de développement d'un projet informatique, depuis l'analyse des besoins jusqu'à la conception, l'implémentation et les tests.

L'utilisation de l'architecture MVC a permis d'organiser le code de manière claire en séparant la partie interface, la logique de traitement et la gestion des données.

Ce projet nous a également permis d'améliorer nos compétences techniques, notre capacité à résoudre des problèmes et notre travail en équipe.

Enfin, cette expérience représente une étape importante dans notre apprentissage du développement d'applications et nous a permis de mieux comprendre la conception d'une application complète.

Références

[1] Documentation officielle Java - Oracle

<https://docs.oracle.com/en/java/>

[2] Documentation officielle JavaFX

<https://openjfx.io/>

[3] Documentation MySQL

<https://dev.mysql.com/doc/>

[4] Documentation JDBC - Oracle

<https://docs.oracle.com/javase/tutorial/jdbc/>

[5] Documentation PlantUML (création des diagrammes UML)

<https://plantuml.com/>